

Proyecto Fin de Mater

Ingeniería y Desarrollo de Soluciones de IA Generativa



Sistema Speech Analytics

Sergio Polo Garcia

Índice

Resumen de la iniciativa	2
Introducción	3
Contexto del problema	4
Objetivos generales.....	4
Objetivos técnicos.	4
Diseño de la solución	4
Arquitectura	5
Recursos compartidos	5
Microsoft Foundry (Azure)	6
MCP.....	6
Almacenamiento de datos.....	7
Módulo de indexación	8
Módulo de consultas.....	9
Cliente	10
Servidor	10
Código	11
Prueba de funcionamiento.....	12
Conclusiones	12
Bibliografía y recursos de interés	14

Resumen de la iniciativa

El siguiente proyecto se centra en implementar una solución Python que trabajará en el callcenter de una empresa de alarmas con dos objetivos principales:

1. Analizar las transcripciones de las llamadas recibidas extrayendo:
 - Motivo de la llamada
 - Categoría
 - Sentimiento del cliente
 - Educación del operador
 - Resolución (booleano)
 - Dispositivos mencionados
 - Resumen
 - Confidence (métrica de control de calidad del proceso)
2. Disponibilizar la información al usuario de negocio mediante un ChatBot agentico que analizará a los resultados del proceso mencionado en el punto anterior.

Introducción

Contexto del problema.

Los usuarios del negocio tardan mucho en obtener métricas de calidad fiables del callcenter.

El método actual de auditoria depende de:

1. Escuchas realizadas por los jefes de equipo.
 - Se estima que en un callcenter promedio solo se pueden auditar entre el 3% y el 5% de las llamadas.
 - Los jefes de equipo invierten gran parte de su jornada a realizar escuchas en lugar de apoyar al empleado o realizar tareas de BackOffice.
2. Clasificación realizada por los empleados que atienden al teléfono en el sistema de gestión de llamadas.
 - Los empleados muchas veces no clasifican bien los motivos de llamada (por velocidad de la operativa o por desconocimiento de las categorías)
3. Encuestas de satisfacción enviadas al cliente para obtener el NPS.
 - Estas encuestas rara vez son respondidas, por lo general suelen tener más respuestas de los clientes descontentos que buscan reclamar.

Objetivos generales.

- Reducir la carga de trabajo de los jefes de equipo y facilitar la toma de decisiones.
- Aumentar el porcentaje de llamadas auditadas.

Objetivos técnicos.

- Practicar la programación de un servidor MCP.
- Probar el framework de agentes “Microsoft Agent Framework”.
- Probar el protocolo AG-UI (Agent-User Interaction) para la interacción usuario-agente.

Diseño de la solución

Arquitectura

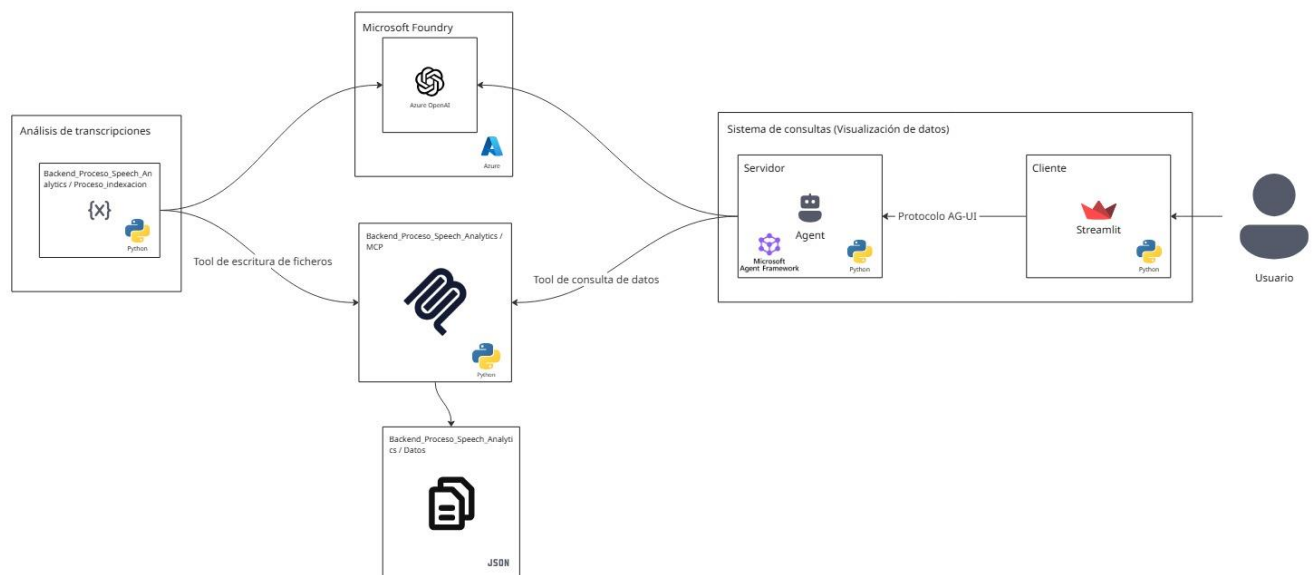
La solución actualmente solo puede ejecutarse en local, consiste en la siguiente estructura de carpetas:

TFM (raíz)

- Backend_Proceso_Speech_Analytics
 - o Proceso_indexacion
 - o MCP
 - o Datos
- Sistema_Consultas
 - o Cliente
 - o Servidor

Cada una de estas carpetas contiene un fichero README.md con información de su función, los paquetes necesarios para su instalación y el comando de ejecución.

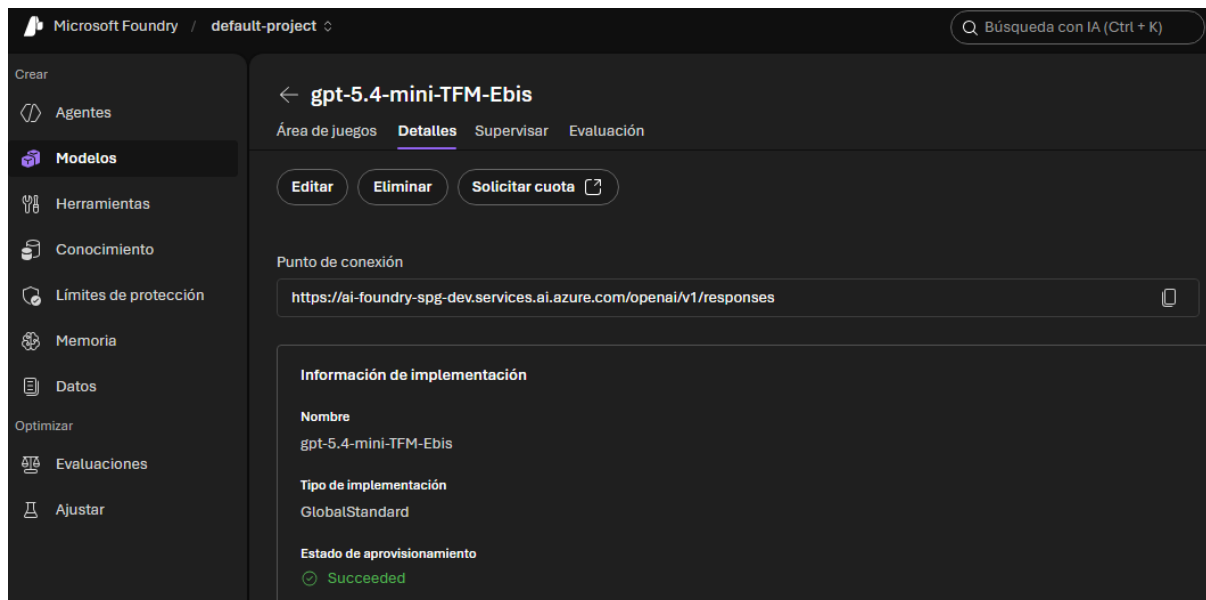
En lo que a arquitectura del proceso se refiere, se seguiría el siguiente diagrama.



Recursos compartidos

Microsoft Foundry (Azure)

Se ha creado un tenant en la nube de Microsoft Azure, dentro de este se ha instanciado una implementación de Microsoft Foundry, y, dentro de esta se ha creado una instancia del modelo de OpenAi GPT-5.4-mini.



Los accesos a este modelo de inteligencia artificial se realizan por dos métodos diferentes:

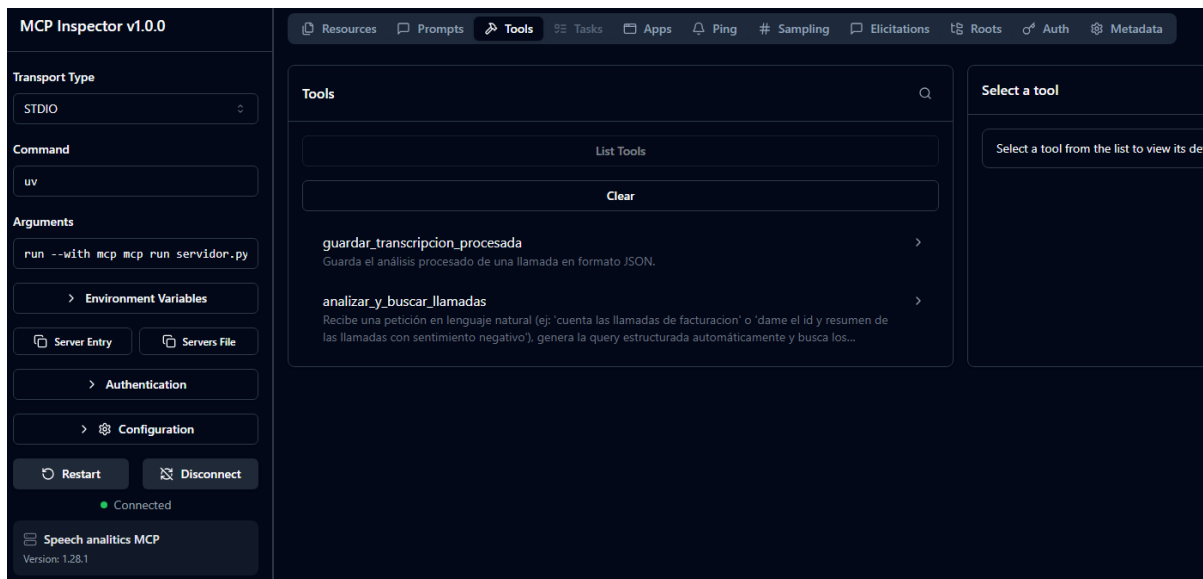
- Acceso por Api Key
 - o [MCP](#)
 - o [Módulo de indexación.](#)
- Acceso por Credenciales de Azure.
 - o [Módulo de consultas \(servidor\)](#)

MCP

MCP desarrollado en Python, expone tools para los módulos de indexación y de consulta.

Tools expuestas

- guardar_transcripcion_procesada
 - Recibe el resultado del [análisis de la transcripción](#) (JSON) y lo almacena en la [carpeta de datos](#).
- analizar_y_buscar_llamadas
 - Recibe una pregunta en lenguaje natural, se encarga de transformarla a un filtro estructurado, posteriormente ejecuta ese filtro sobre la [carpeta de datos](#) y devuelve el resultado de la búsqueda.
 - Esta tool tiene conexión con Microsoft Foundry para utilizar una implementación de Azure OpenAI.



Almacenamiento de datos

Carpeta alojada en “Backend_Proceso_Speech_Analytics/Datos”, se encarga de almacenar los resultados del [módulo de indexación](#).

Actualmente almacena los resultados del proceso en forma de ficheros JSON (1 por transcripción), utilizando la siguiente estructura:

```
{
  "id": INT,
  "motivo": STR,
  "categoria": STR,
  "sentimiento_cliente": STR,
  "educacion_operador": FLOAT,
  "resolucion": BOOL,
  "dispositivos_mencionados": STR [],
  "resumen": STR,
  "confidence": {
    "motivo": FLOAT,
    "categoria": FLOAT,
    "sentimiento_cliente": FLOAT,
    "educacion_operador": FLOAT,
    "resolucion": FLOAT
  }
}
```


Módulo de indexación

Módulo encargado de recibir la transcripción de la llamada, procesarla y guardar su resultado.

Componentes

- Script de Python.
- Carpeta “transcripciones_llamadas”.
- Fichero “prompt.txt”.

Proceso

Las transcripciones han de alojarse en ficheros .txt separados y en la carpeta “transcripciones_llamadas”, posteriormente se debe ejecutar el script de Python de forma manual, este ejecutará las siguientes acciones:

1. Al iniciarse, realiza una conexión con el [MCP](#) para extraer sus tools.
2. Accede a carpeta “transcripciones_llamadas” y extrae los ficheros txt.
3. Crea la conexión con el cliente de Azure OpenAI mediante Api Key.
4. Extrae el prompt del fichero “prompt.txt” y lo parametriza con la transcripción a analizar.
5. Ejecuta la consulta a Azure OpenAI.
6. El resultado de esta consulta es un JSON, se revisa su estructura y ejecuta la tool “guardar_transcripcion_procesada” del MCP para almacenar el resultado.

Prompt

Se ha escrito en formato Markdown y se ha estructurado de la siguiente manera:

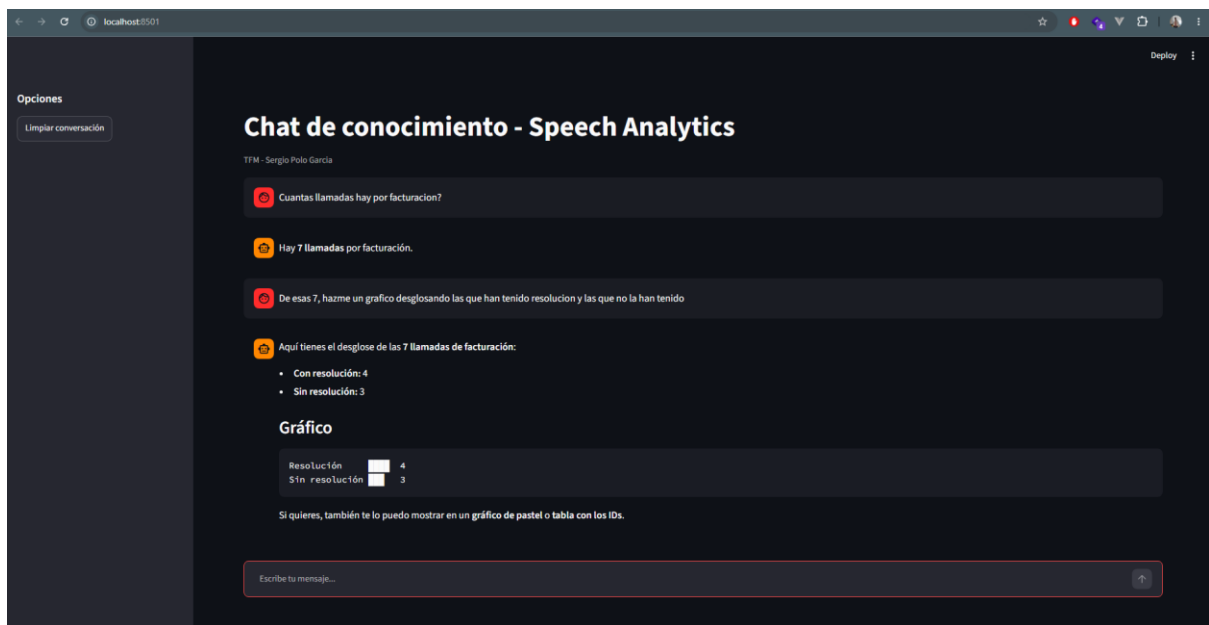
1. Definición del rol que debe adoptar el modelo.
2. Instrucciones.
3. Definición del esquema de salida (json).
4. Definición de los campos y valores permitidos.
5. Inyección de la transcripción (mediante variable “{{TRANSCRIPCION}}”).

Módulo de consultas

Cliente

Chat presentado al usuario para que realice preguntas en lenguaje natural sobre los datos almacenados en el [módulo de datos](#).

Este módulo está desarrollado en Python y utiliza Streamlit, para la conexión con el [Módulo de consultas \(servidor\)](#) utiliza el protocolo AG-UI.



Servidor

Agente hecho en Python utilizando el framework de Microsoft para el desarrollo Agentes ([Microsoft Agent Framework](#)), al iniciarse obtiene las tools que haya disponibles en el [módulo MCP](#).

El agente mantiene una conversación con el usuario en lenguaje natural (a través de conexión con el cliente enlazado mediante el protocolo AG-UI), al detectar una pregunta relacionada con la operativa del callcenter, utiliza la tool “analizar_y_buscar_llamadas” para convertirla a un filtro estructurado y realizar una búsqueda eficiente.

Una vez obtiene el resultado para la pregunta, si lo cree conveniente (o se lo pide el usuario), genera un gráfico para representar la respuesta, en el resto de los casos, devuelve la respuesta en texto plano.

Tecnologías aplicadas

- **Microsoft Agent Framework**

Framework desarrollado por Microsoft para crear agentes de inteligencia artificial. De los aspectos más relevantes de este Framework es su integración con el ecosistema Microsoft, el framework permite utilizar servicios como Azure OpenAI, Microsoft Entra ID para la autenticación o incluso Azure Functions.

- **AG-UI (Agent-User Interaction Protocol)**

Protocolo desarrollado por la comunidad de CopilotKit, su objetivo es definir un estándar de comunicación entre los agentes IA y las interfaces de usuario, permitiendo que cualquier frontend pueda interactuar con diferentes frameworks de agentes mediante un protocolo común.

El uso de este protocolo permite que un mismo cliente puede conectarse a distintos agentes sin modificar su implementación.

Código

Puede encontrarse el código de todos los módulos de la solución en el siguiente repositorio de GitHub, cada módulo/carpeta tiene un documento README.md con las instrucciones para iniciarlo.

LINK: https://github.com/SergioPoloGarcia/TFM_Ebis.git

Prueba de funcionamiento

Tras el procesamiento de 100 transcripciones, se ha ejecutado el módulo de consultas para realizar una prueba de como un usuario conversaría con el chat, se adjunta evidencia en video de los resultados en el repositorio de GitHub donde se encuentra el código.

Conclusiones

El desarrollo de este proyecto me ha permitido crear una solución basada en IA generativa orientada a la automatización del análisis de llamadas en un entorno de callcenter. La solución desarrollada aborda una problemática real del negocio como es la dificultad para obtener métricas fiables y completas sobre la atención ofrecida a los clientes debido a las limitaciones del proceso tradicional de auditoría manual.

Mediante la implementación del módulo de indexación se ha conseguido realizar el análisis de las transcripciones de llamadas, esto permite aumentar significativamente el volumen de llamadas analizadas, superando las limitaciones del modelo tradicional basado en escuchas manuales.

Uno de los principales logros técnicos del proyecto ha sido la implementación de un servidor MCP (Model Context Protocol) desarrollado en Python, este componente actúa en común con ambos módulos de la solución, exponiendo tools tanto para el proceso de indexación como para el agente conversacional.

La integración de Microsoft Agent Framework y del protocolo AG-UI ha permitido desarrollar un agente conversacional capaz de interactuar con usuarios mediante lenguaje natural a través de un frontal sustentado en Streamlit. Este enfoque mejora la accesibilidad de la información generada, permitiendo que perfiles de negocio puedan consultar los datos analizados sin necesidad de conocimientos técnicos ni de realizar consultas manuales.

Desde el punto de vista práctico, la solución desarrollada supone una evolución del modelo tradicional de control de calidad del callcenter, la automatización del análisis permite reducir la carga operativa de los jefes de equipo, liberando tiempo para tareas de mayor valor y facilitando la toma de decisiones.

Durante el desarrollo también se han identificado diferentes líneas de evolución necesarias para convertir el prototipo/prueba de concepto actual en una solución preparada para un entorno productivo, entre ellas me gustaría destacar:

- El despliegue de los componentes en servicios cloud (azure).
- La sustitución del almacenamiento basado en ficheros JSON por sistemas como BBDD o Data lakes.
- La automatización del proceso de indexación mediante servicios gestionados como Azure Functions o Databricks.
- La integración con servicios como Genesys para la recepción automática de transcripciones.
- El uso de la funcionalidad Batch de Azure OpenAI para abaratar costes de procesamiento.

Bibliografía y recursos de interés

Durante el desarrollo del proyecto se han consultado las siguientes fuentes y recursos técnicos relacionados con IA generativa, desarrollo de agentes, protocolos de comunicación y servicios cloud:

- Microsoft Azure AI Foundry.
<https://learn.microsoft.com/azure/ai-foundry/>
- Azure OpenAI Service.
<https://learn.microsoft.com/en-us/azure/foundry/openai/reference>
- Microsoft Agent Framework.
<https://learn.microsoft.com/es-es/agent-framework/overview/?pivots=programming-language-python>
- Model Context Protocol (MCP).
<https://modelcontextprotocol.io/>
- AG-UI Protocol.
<https://www.ag-ui.com/>
- Streamlit.
<https://docs.streamlit.io/>
- Raisetalk. Quality Monitoring en centros de callcenter.
<https://www.raisetalk.com/es/blog/quality-monitoring-centros-llamadas-guia-completa>